



Phage
SECURITY

SECURITY REVIEW · PREPARED FOR

Alt Fun

A permissionless token
launchpad on HyperEVM

DATE

14.05.2026

CONDUCTED BY

Pyro, Lead Security Researcher
BengalCatBalu, Lead Security Researcher
Kvarr, Security Researcher

COMMIT

474837d

REMEDIATION

f7f953b

01 ABOUT

Phage Security

Phage Security is a team of highly skilled smart-contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike.

With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

PREVIOUS WORK

github.com/phage-security

REACH OUT

Telegram · @Pyro3b

WHAT AN AUDIT IS

An audit is a time, resource, and expertise-bound effort in which trained researchers evaluate smart contracts using a combination of manual and automated techniques to identify as many vulnerabilities as possible.

An audit can reveal the presence of vulnerabilities, but it cannot guarantee their absence. This report is a point-in-time assessment of the code at the commits named on the cover. It is not a warranty that the system is secure.

02 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	ACKNOWLEDGED
10	8	2

SEVERITY DISTRIBUTION



CLIENT	REPOSITORY	METHODS
Alt Fun	Private	Manual review

CONTRACTS IN SCOPE

packages/contracts/src/Bonding.sol	packages/contracts/src/Factory.sol
packages/contracts/src/FeeVault.sol	packages/contracts/src/LPLOCK.sol
packages/contracts/src/Pair.sol	packages/contracts/src/Router.sol
packages/contracts/src/Token.sol	packages/contracts/src/Zap.sol
packages/contracts/src/lib/VanityMining.sol	

TIMELINE · MAY 2026

Audit · 4 days		Remediation · 6 days	
KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
03.05.2026	06.05.2026	08.05.2026	13.05.2026

STARTING COMMIT HASH	474837dbd1e508423431c816fc2a8eb817aa3e89
FINAL COMMIT HASH	f7f953bd173fc51e3dc920b19a34f0875e86904a

03 PROJECT SUMMARY

What Alt Fun is

Alt Fun is a permissionless token launchpad on HyperEVM. Anyone can launch a new ERC20 paired with a BounceTech Leveraged Token as the curve's reserve, and users trade in USDC the whole way through.

Each token starts on an internal bonding curve and graduates to a HyperSwap V2 pool once it crosses a USD threshold (or all of the tokens are sold), with the resulting LP locked permanently. A small fee is skimmed on every buy and sell and split between the token's creator and the protocol.

The codebase came in unusually clean. Only two findings reached Medium severity and the rest were Low, with no Highs and no Criticals, and the core curve, graduation, and LP-locking logic held up well under review. Most of what the audit surfaced sat at the seams between the launchpad and the BounceTech LT it wraps, not in the core flow itself.

SECURITY POSTURE

BEFORE REMEDIATION

474837d

Cross-graduation accounting

M-02 · L-04

When two tokens shared the same LT, residual LT from one graduation could be silently consumed by the next, and pair donation skims routed unsolicited transfers into `LPLock`, which has no rescue path.

Money-flow rounding in Zap

L-02 · L-05 · L-07

Closing buys overshot the curve and paid an avoidable redemption fee, the min-amount check ran on gross instead of net USDC, and the post-graduation fee policy was misdocumented.

Smaller liveness and design notes

M-01 · L-01 · L-03 · L-06 · L-08

Mint and redeem pause asymmetry, a missing `deadline` on Zap, and a vanity probe that could OOG `setTokenImplementation`, plus two design points flagged for documentation: graduation trigger semantics and the launch-time exchange-rate snapshot.

AFTER REMEDIATION

f7f953b

Eight of ten findings fixed and verified. The Alt Fun team shipped code fixes for M-02, L-02, L-04, L-05, and L-08, and closed M-01, L-01, and L-07 with documentation fixes that pin down the intended behavior. Several patches went beyond the literal bug: the cross-graduation fix introduces a `protectedLT` snapshot and a new `_sweepLTtoOwner` post-bookend that also resolves the donation skim case, closing M-02 and L-04 in one pass, and the closing-buy fix adds a new `Router.previewBuy` view to pre-size the LT mint and refund USDC directly.

The two acknowledged items, L-03 and L-06, reflect deliberate design choices: the launch-time exchange-rate snapshot is an accepted property of the curve, and the missing Zap deadline carries little risk with HyperEVM's near-immediate inclusion. The codebase is in good shape for mainnet.

04 RECOMMENDATIONS

Beyond this audit

Four measures that extend the protection of this review after launch, independent of any individual finding.

R-01	Bug bounty	Set up a bug bounty program to incentivize ongoing third-party review beyond the audit window.
R-02	Invariant monitoring	Track <code>FeeVault.totalAccruedCreator + protocolBalance</code> vs. the vault USDC balance, <code>LPLock.locks(token).amount</code> vs. the actual LP balance held at the lock, and <code>LTRescued</code> event volume vs. expected concurrent-graduation flow, and page on any divergence.
R-03	Owner key hygiene	Document multisig signing-service policies, rotation procedure, and an incident-response runbook for compromised owner keys across <code>Bonding</code> , <code>Zap</code> , <code>FeeVault</code> , and <code>LPLock</code> . The <code>Ownable2StepUpgradeable</code> two-step transfer covers fat finger errors but not key compromise.
R-04	Anomaly dashboards	Surface graduation rebalance utilization (Regime 3 hits per period), LP seeding off-ratio remainder, capped buy USDC refund volume, per-LT mint-pause periods, and <code>FeeVault.sweepDonations</code> activity for community and internal review.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
<code>packages/contracts/src/Bonding.sol</code>	Central state machine for the per-token lifecycle from Curve through Graduating to Graduated. Holds the LP-seeding logic with hostile pre-seed defense and the immutable <code>graduationThresholdUsd</code> .
<code>packages/contracts/src/Router.sol</code>	Bonding-curve AMM math, constant product with virtual reserves, no fees. State-mutating entry points are gated by <code>BONDING_ROLE</code> .
<code>packages/contracts/src/Pair.sol</code>	Per-token bonding curve pair, reserves not sorted by address. K is pinned at <code>mint</code> and only checked in <code>swap</code> .
<code>packages/contracts/src/Factory.sol</code>	Registry of bonding curve pairs. <code>setRouter</code> is one-shot, after the first pair the router cannot be rotated.
<code>packages/contracts/src/Token.sol</code>	ERC20 launchpad token with a fixed 1B supply and EIP-2612 permit, cloned via EIP-1167 minimal proxies. <code>Bonding</code> is the only burner.
<code>packages/contracts/src/Zap.sol</code>	User-facing entry point routing USDC to tokens via LT mint and redeem plus curve or V2 swaps, skimming fees on every trade to <code>FeeVault</code> . Enforces a 10 USDC minimum on <code>netUsdc</code> .
<code>packages/contracts/src/FeeVault.sol</code>	Holds creator and protocol USDC fees from allowlisted depositors, which transfer USDC and then call <code>accrue</code> . Permissionless <code>sweepDonations</code> routes unbacked USDC to <code>feeTo</code> .
<code>packages/contracts/src/LPlock.sol</code>	Locks LP tokens at graduation, UUPS upgradeable, no withdraw in v1. The locker allowlist is add-only, retiring one requires an upgrade.

All contracts are upgradeable except Pair, Factory, and Token. VanityMining is an internal library.

PRIVILEGED ROLES

Owner (multisig). Owns `Bonding`, `Zap`, `FeeVault`, and `LPlock` behind a two-step `Ownable2StepUpgradeable` transfer. Rotates fees, the token implementation, and the BounceTech `GlobalStorage` mirror.

BONDING_ROLE. Held by `Bonding` only. Gates every router-side entry point behind the lifecycle gates and the `Bonding` -level reentrancy guard.

Locker. Allowlisted callers of `LPlock.recordLock`. Held by `Bonding` in v1.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
M-01	<code>Zap.sol :: buy()</code> will get DoSed when LT minting is paused, creating a sell-only market	MEDIUM	● FIXED
M-02	Leftover LT from one graduation can be reused in another token's LP Pool	MEDIUM	● FIXED
L-01	Graduation only fires on buy, contradicting "Immediate Trigger" documented behavior	LOW	● FIXED
L-02	Closing buyer pays LT redemption fee on Leverage Token redeem	LOW	● FIXED
L-03	Launch-Time exchange-rate snapshot permanently shapes the bonding curve	LOW	● ACKNOWLEDGED
L-04	Donated assets in LP pair are skimmed to <code>LPLock</code> and become stuck	LOW	● FIXED
L-05	Buy min-amount check validates gross USDC instead of net mint input	LOW	● FIXED
L-06	Missing deadline on Zap trade functions	LOW	● ACKNOWLEDGED
L-07	Fee is charged on post graduation trades	LOW	● FIXED
L-08	Vanity probe in <code>setTokenImplementation</code> will OOG and brick impl rotation	LOW	● FIXED

● MEDIUM SEVERITY · 2 FINDINGS

M-01

MEDIUM

FIXED

Zap.sol::buy() will get DoSed when LT minting is paused, creating a sell-only market

LeveragedToken is only mint-pausable, while redeems cannot be paused. If an LT gets mint-paused, any launched token using that LT has `buy()` DoSed, because `Zap` has to call `lt.mint()` :

```
IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
```

However, `sell()` still works, since it uses `lt.redeem()` :

```
IBounceLeveragedToken(lt).redeem(address(this), ltReceived, 0);
```

Additionally, for graduated tokens, `Zap` still mints LT on `buy()` because users input USDC. So if LT minting is paused, `Zap.sol::buy()` still reverts even though the HyperSwap `TOKEN/LT` pool itself is live. Users who already have LT could still buy directly through HyperSwap outside of `Zap`, without minting LT.

IMPACT

During the curve period, a mint-paused LT makes the market sell-only. Users can exit, but new buyers cannot enter, so holders may rush to sell before others do, knowing buys are unavailable until minting is unpaused.

There is also a graduation side effect: if the LT exchange rate makes `canGraduate()` return true while minting is paused, the token cannot enter the Graduating state, since that transition only happens inside `Bonding.sol::buy()` and buys are unavailable.

RECOMMENDATION

For the curve period, there are a few possible choices:

1. Acknowledge and document the behavior. If LT minting is paused, it may still be acceptable to let users sell and exit to USDC.
2. If LT minting is paused and buying is DoSed, also block selling.
3. Add a way for the token creator to pause sells when the LT is mint-paused. This should only be possible while LT minting is paused, so creators cannot pause sells arbitrarily.

After graduation, consider adding a way to buy through `Zap` with already-held LT, so users can still access the `TOKEN/LT` pool even when new LT minting is paused.

TEAM

Fixed via documentation in alt-fun PR #369. The team chose option 1 (document the behavior, accept the asymmetry).

M-02

MEDIUM

FIXED

Leftover LT from one graduation can be reused in another token's LP Pool

When `finalizeGraduation()` is called, if the HyperSwap pair already has liquidity, the flow goes through `_seedRebalancing()`. If the attacker seeded the pool making it LT-rich, `Bonding` swaps `token` → `LT` to fix the ratio. After `addLiquidity()`, this can leave leftover LT in `Bonding`. This is known, since `Bonding.sol::rescueLT()` exists to rescue LT in such cases.

However, if another token using the same LT graduates later, an attacker can preseed that second pool in the opposite direction before `finalizeGraduation()`, making it token-rich. In that case, the rebalance swaps `LT` → `token`. Normally this should leave leftover tokens, which are burned in `_routerDepositAndDispose()`.

The issue is that `_routerDepositAndDispose()` checks the available LT using `IERC20(lt).balanceOf(address(this))`. This assumes the LT balance belongs to the current graduation, but it also includes LT left in `Bonding` from previous graduations using the same LT. So instead of the owner later calling `rescueLT()`, that previous leftover LT can be used as liquidity for another token's pool.

IMPACT

An attacker can cause the following: Token A and Token B both use the same LT and are ready to graduate.

1. Create/preseed the HyperSwap pool for Token A, making it LT-rich.
2. `finalizeGraduation(A)` leaves leftover LT in `Bonding`.
3. Create/preseed the HyperSwap pool for Token B, making it token-rich.
4. `finalizeGraduation(B)` enters `_routerDepositAndDispose()`. Normally the imbalance would leave leftover Token B, which gets burned. However, because the function uses the full LT balance of `Bonding`, the leftover LT from Token A is also used and deposited into Token B's LP instead of being rescued.

This is medium severity because the attacker may not be able to directly steal the LT, but it breaks accounting between different tokens using the same LT. Token A's leftover LT is moved into Token B's liquidity, Token B gets extra liquidity, and the owner loses LT that should have been recoverable through `rescueLT()`.

RECOMMENDATION

In `_routerDepositAndDispose()`, any leftover LT should be moved to a dedicated recipient or accounted separately, so later graduations using the same LT cannot consume it.

TEAM

Fixed in alt-fun PR #371. The team went beyond the original recommendation. `finalizeGraduation` now snapshots `protectedLT = balanceOf(this) - p.ltFromPair` before any LP work and threads it through `_routerDepositAndDispose` so the deposit allowance is sized off (`ltBal - protectedLT`). The new `_sweepLTtoOwner` helper transfers any remaining residue to the owner with an `LTRescued` event, so honest graduations no longer accumulate stuck LT either.

● LOW SEVERITY · 8 FINDINGS

L-01

LOW

FIXED

Graduation only fires on buy, contradicting "Immediate Trigger" documented behavior

docs/contracts-scope.md:73 states graduation is "executed in `Bonding._graduate()` immediately when true". The implementation only enters phase 1 from inside `_executeBuy`. `_enterGraduating` is internal with no external entry point:

```
// Bonding.sol - only call site
if (canGraduate(tokenAddress)) {
    _enterGraduating(tokenAddress);
}
```

`canGraduate`'s USD path reads live `exchangeRate()`, so a token can become ripe purely from a rate move. With no buy, the ripe state can be lost before it's captured (rate retrace, or a sell pulling `realLtRaised` back below threshold).

IMPACT

Tokens that cross the USD threshold without an accompanying buy may never graduate. Behavior diverges from the documented "immediate" trigger.

RECOMMENDATION

Either fix the docs to say phase 1 only fires on the threshold-crossing buy, or expose a permissionless poke:

```
function pokeGraduate(address token) external {
    if (!canGraduate(token)) revert NotGraduable();
    _enterGraduating(token);
}
```

TEAM

Fixed via documentation in alt-fun PR #367. The team opted to keep the contract surface unchanged and update the docs to clarify graduation only fires on the threshold crossing buy. A keeper bot will trigger immediate graduations off-chain via a small buy when the USD-only threshold is crossed without organic flow, so end-user behavior matches the original "immediate" intent without adding a permissionless on-chain entry point.

L-02

LOW

FIXED

Closing buyer pays LT redemption fee on Leverage Token redeem

`Zap._executeBuy` mints LT from the full `netUsdc` before knowing how much the curve will actually accept. On a capped buy (graduating tx, when `_computeBuy` hits `tokensOut > realBalance`), the leftover LT is redeemed back to USDC for the user, and `LT.redeem` charges `percentFee = baseAmount * redemptionFee * targetLeverage` on that leg:

```
// Zap.sol - _executeBuy
uint256 ltMinted = IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
// ... curve consumes only amountInUsed, leaving ltLeft
uint256 ltLeft = ltMinted - amountInUsed;
if (ltLeft > 0) {
    IBounceLeveragedToken(lt).redeem(msg.sender, ltLeft, 0); // pays redemptionFee
}
```

The round-trip is avoidable: `Router._computeBuy` is a `view` and returns the exact `amountInUsed` for any input. `Zap` could pre-size the mint to `ltToBaseAmount(amountInUsed)` and refund the unconverted USDC directly, leaving no LT to redeem and no fee to pay.

IMPACT

The closing buyer of every graduation pays an avoidable LT redemption fee on the overshoot.

RECOMMENDATION

Pre-size the mint against `_computeBuy` instead of overshoot-and-refund:

```
uint256 ltMinted = IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
(tokensOut, amountInUsed) = _buyOnCurve(tokenAddress, lt, ltMinted);
uint256 ltLeft = ltMinted - amountInUsed;
if (ltLeft > 0) IBounceLeveragedToken(lt).redeem(msg.sender, ltLeft, 0);
+ uint256 ltTarget = IBounceLeveragedToken(lt).baseToLtAmount(netUsdc);
+ (uint256 ltNeeded,) = router.previewBuy(tokenAddress, ltTarget);
+ uint256 baseToConvert = IBounceLeveragedToken(lt).ltToBaseAmount(ltNeeded);
+ IBounceLeveragedToken(lt).mint(address(this), baseToConvert, 0);
+ (tokensOut, amountInUsed) = _buyOnCurve(tokenAddress, lt, ltNeeded);
+ uint256 usdcLeft = netUsdc - baseToConvert;
+ if (usdcLeft > 0) $.usdc.safeTransfer(msg.sender, usdcLeft);
```

TEAM

Fixed in alt-fun PR #370. A new public `Router.previewBuy` `view` was added (returning `(amountInUsed, tokensOut)` for any LT-in input), and `Zap._executeBuy` now pre-sizes the LT mint to the exact needed amount via `previewBuy(ltIfFull)` and `ltToBaseAmount(ltNeeded)`. Any unconverted USDC is refunded directly to the buyer instead of being round tripped through `LT.redeem`, so the closing buyer no longer pays the `baseAmount * redemptionFee * targetLeverage` surcharge on the overshoot. The fee proration was also tightened to charge only on the actual `baseToConvert` portion, with the unused fee refunded.

L-03

LOW

ACKNOWLEDGED

Launch-Time exchange-rate snapshot permanently shapes the bonding curve

`Bonding._deployAndSeed` derives `virtualLtReserve` from a single read of `LT.exchangeRate()`, and `Pair.mint` bakes it into $k = \text{TOTAL_SUPPLY} * \text{virtualLtReserve}$ for the lifetime of the curve:

```
// Bonding.sol - _deployAndSeed
uint256 exchangeRate = IBounceLeveragedToken(ltAddress).exchangeRate();
uint256 virtualLtReserve = (VIRTUAL_LIQUIDITY_USD * 1e18) / exchangeRate;
$.router.addInitialLiquidity(tokenAddr, totalSupply, curveSupply, virtualLtReserve);
```

A higher snapshot means a lower `virtualLtReserve`, a lower `k`, and a larger price impact per LT in. A lower snapshot means the opposite. The creator picks `ltAddress` but has no parameter to bound the rate they accept, and `exchangeRate` is donation-sensitive (`baseAssetBalance` counts direct USDC transfers). Whatever transient state the LT is in at the inclusion block, a donation, bridging, or a PnL spike, defines the curve forever.

IMPACT

Every future buy/sell price along the curve is anchored to a single externally-mutable read taken at launch. If the `K` parameter is set to incorrect values, it directly affects the token's future pricing.

RECOMMENDATION

Let the creator pass a min/max band for the accepted rate, mirroring AMM slippage:

```
struct LaunchParams {
    ...
+   uint256 minExchangeRate;
+   uint256 maxExchangeRate;
}
```

```
uint256 exchangeRate = IBounceLeveragedToken(ltAddress).exchangeRate();
+ if (exchangeRate < params.minExchangeRate || exchangeRate > params.maxExchangeRate) {
+   revert ExchangeRateOutOfBand();
+ }
uint256 virtualLtReserve = (VIRTUAL_LIQUIDITY_USD * 1e18) / exchangeRate;
```

TEAM

Acknowledged via documentation in alt-fun PR #372. The team rejected the (`min`, `max`) band proposal, as it would introduce a launch failure mode users can't see and forces the frontend into a tolerance that's either too tight (legitimate launches fail) or too loose (decorative).

L-04

LOW

FIXED

Donated assets in LP pair are skimmed to LPLock and become stuck

When finalizing a graduation, `_seedUniswapV2Direct()` first calls `IUniswapV2Pair(pair).skim(_s().lpLock)`. This means that if the HyperSwap pair has any donated balances above reserves, they are skimmed to `LPLock`. This is incorrect for both assets.

1. If the skimmed asset is LT, it gets sent to `LPLock` and is effectively stuck there. `LPLock` has no rescue functionality, even though the LT has actual value.
2. If the skimmed asset is the launched token, it also gets stuck in `LPLock` instead of being burned. This is inconsistent with the rest of the graduation flow, where unused tokens are burned. For example, `_routerDepositAndDispose()` burns `leftoverToken` after the router liquidity deposit.

Additionally, if the LP pool already has reserves, the flow later goes through `_seedRebalancing()` and `_routerDepositAndDispose()`. In that case, the skimmed assets could theoretically be kept in `Bonding` and used in the liquidity deposit, with any leftover TOKEN burned and any leftover LT left available for `rescueLT()`.

IMPACT

Donated LT can become permanently stuck in `LPLock` instead of being used or rescued. Donated launched tokens can also become stuck in `LPLock` instead of being burned, leaving supply that the rest of the graduation flow would normally remove.

RECOMMENDATION

Do not skim donated balances to `LPLock`. Skim them to `Bonding` instead, then let `_routerDepositAndDispose()` handle them: use what can be used for liquidity, burn leftover TOKEN, and leave leftover LT available for `rescueLT()`.

TEAM

Fixed in alt-fun PR #381 with the recommended approach. `_seedUniswapV2Direct` now skims donations to `address(this)` instead of `LPLock`.

L-05

LOW

FIXED

Buy min-amount check validates gross USDC instead of net mint input

`MIN_USDC_AMOUNT = 10e6` is pre-checked on the buy path so users see `BelowMinAmount` instead of the LT's undecodable `0x05eb05ac` (`BelowMinTransactionSize`) selector. The buy path checks the wrong value: `_buyInternal` validates the gross `usdcAmount`, but `_executeBuy` deducts the fee and forwards `netUsdc` to `LeverageToken.mint`, which is what the LT actually validates against its floor.

```
// Zap.sol - _buyInternal
if (usdcAmount < MIN_USDC_AMOUNT) revert BelowMinAmount();
```

```
// Zap.sol - _executeBuy
uint256 netUsdc = usdcAmount - feeOnGross;
uint256 ltMinted = IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
// ^ netUsdc, not usdcAmount, is what hits the LT floor
```

Any `usdcAmount` in `[MIN_USDC_AMOUNT, MIN_USDC_AMOUNT / (1 - buyFeeBps/BPS_DENOM)]` passes the Zap check and reverts inside `mint` with the exact selector the pre-check was added to suppress.

RECOMMENDATION

Validate the value forwarded to `mint`:

```
uint256 netUsdc = usdcAmount - feeOnGross;
+ if (netUsdc < MIN_USDC_AMOUNT) revert BelowMinAmount();
$.usdc.forceApprove(lt, netUsdc);
uint256 ltMinted = IBounceLeveragedToken(lt).mint(address(this), netUsdc, 0);
```

Alternatively, rework the money flow and deduct fees after calling `lt.mint`.

TEAM

Fixed in alt-fun PR #378 with the recommended approach.

L-06

LOW

ACKNOWLEDGED

Missing deadline on Zap trade functions

Neither the buy nor the sell functions inside `Zap.sol` accept a `deadline` parameter. `minTokensOut` / `minUsdcOut` bound the magnitude of the loss, but not the time window in which an MEV bot can realize it.

Example (using simple values):

1. User submits `buy(token, 100 USDC, minTokensOut=95)`, current rate is 1:1.
2. TX is delayed a bit (very low likelihood, but possible).
3. Curve rate falls, e.g. from 1.0 to 0.5. The same 100 USDC now mints 200 curve tokens.
4. User's TX executes.
5. MEV bot sees the stale TX in the mempool and sandwiches it (buy to push the curve back to 1:1, plus sell back to 1:0.5).

The user got their 5% slippage against the original rate, against the executed rate they paid ~50% over fair value. The same can happen with `sell`.

HyperEVM has MEV, however the potential for this attack is small since most TXs are included in the block in which they are made.

RECOMMENDATION

Consider adding a `deadline` to `buy/sell` (and their permit versions):

```
function buy(
    address tokenAddress,
    uint256 usdcAmount,
    uint256 minTokensOut,
    address referrer,
+   uint256 deadline
) external nonReentrant returns (uint256 tokensOut) {
+   if (block.timestamp > deadline) revert DeadlineExpired();
    return _buyInternal(tokenAddress, usdcAmount, minTokensOut, referrer);
}
```

TEAM

Acknowledged. The team noted that HyperEVM's typical inclusion behavior keeps the practical attack window narrow, so the `deadline` parameter was not added in this remediation round.

L-07

LOW

FIXED

Fee is charged on post graduation trades

The provided docs show this table:

FEE	RATE	SPLIT
Router buy	0.5%	0.4% protocol / 0.1% creator
Router sell	0.5%	0.4% protocol / 0.1% creator
HyperSwap swap (post-grad)	0.3%	HyperSwap LPs
LT redemption	BounceTech-internal	No Alt Fun fee

Per the table, after graduation there should not be an Alt Fun fee on swaps. However the code still charges `buyFeeBps` / `sellFeeBps` :

```
if ($.bonding.isGraduated(tokenAddress)) {
    tokensOut = _buyOnUniswapV2(tokenAddress, lt, ltMinted);
    amountInUsed = ltMinted;
} else {
    (tokensOut, amountInUsed) = _buyOnCurve(tokenAddress, lt, ltMinted);
}
actualFee = ltMinted == 0
    ? 0
    : (usdcAmount * buyFeeBps_ * amountInUsed) / (BPS_DENOM * ltMinted);
```

RECOMMENDATION

Change the code to use 0 fee when the token has graduated:

```
// Zap._executeBuy
- uint256 buyFeeBps_ = $.buyFeeBps;
+ uint256 buyFeeBps_ = $.bonding.isGraduated(tokenAddress) ? 0 : $.buyFeeBps;
uint256 feeOnGross = (usdcAmount * buyFeeBps_) / BPS_DENOM;
// Zap._sellInternal
- uint256 fee = (grossUsdc * $.sellFeeBps) / BPS_DENOM;
+ uint256 sellFeeBps_ = bonding_.isGraduated(tokenAddress) ? 0 : $.sellFeeBps;
+ uint256 fee = (grossUsdc * sellFeeBps_) / BPS_DENOM;
```

TEAM

Fixed via documentation in alt-fun PR #383. The team confirmed the original docs were misleading: charging fees on every venue (curve and post-graduation) is the intended policy. Lifting fees post-graduation would silently halve protocol and creator revenue the moment a token graduated, which is the opposite of the intent.

L-08

LOW

FIXED

Vanity probe in `setTokenImplementation` will OOG and brick impl rotation

`setTokenImplementation` runs `VanityMining.mine` as a sanity probe before swapping the impl. The miner brute forces a salt with 20 zero trailing bits (~1M iters average per the lib's own comment).

```
VanityMining.mine(address(0x1), bytes32(0), bytes32(0), newImpl, address(this), 0);
```

Per-loop cost is about 150 gas. With HyperEVM's big blocks capped at 30M gas, the probe must converge in under 200k loops to fit, so ~83% of impls OOG.

Since the salt sequence is deterministic for `(0x1, 0, 0, newImpl, this, 0)`, retrying the same `newImpl` always OOGs at the same iter, which means the owner has to redeploy `Token` to fresh addresses until one happens to converge under the cap.

RECOMMENDATION

Consider removing the `VanityMining` part:

```
VanityMining.mine(address(0x1), bytes32(0), bytes32(0), newImpl, address(this), 0);  
address old = $.tokenImplementation;  
$.tokenImplementation = newImpl;
```

TEAM

Fixed in alt-fun PR #385 with the recommended approach. The `VanityMining.mine` probe was removed from `setTokenImplementation` outright. The function now updates `tokenImplementation` directly after the existing `TOTAL_SUPPLY` consistency check.